

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

E-MARS: Emergent Modular Adaptive Reliability System for Microservice Architectures

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Harnoor Kaur

2210990374.

Supervised By:

Dr. Anshu Singla

Professor | Head-Academic Delivery

Chitkara University Rajpura



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iv
	List of Figures and Tables	v
	Abstract	
1	Introduction	1
2	Methodology	
3	Tools and Technologies	
4	Implementation	
5	Major Findings/Outcomes/Output/Results	
6	Conclusion and Future Scope	
	References	
	Appendices	

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**E-MARS: Emergent Modular Adaptive Reliability System for Microservice Architectures**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Anshu Singla. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Harnoor Kaur

Date: 04 May 2026

2210990374

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Anshu Singla

Professor | Head-Academic Delivery

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

The authors would like to thank the faculty and research mentor Dr. Anshu Singla of the Department of Computer Science, Chitkara University, Punjab, India, for her valuable guidance and unwavering support throughout the development of this research project. Her insightful feedback, technical direction, and encouragement were instrumental in shaping the E-MARS framework.

We also acknowledge the open-source communities and research contributions in the areas of microservices architecture, graph-based learning, and site reliability engineering, which provided important conceptual foundations for this work.

Special appreciation is extended to the developers of Python, NetworkX, and Prometheus-based observability tools, whose platforms enabled the prototype implementation and experimental evaluation of the E-MARS framework.

We are also grateful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Tech

LIST OF FIGURES AND TABLES

FIGURE NUMBER	FIGURE NAME
Figure 1	E-MARS System Architecture
Figure 2	Latency vs Workload Comparison (All Four Configurations)
Figure 3	Multi-Metric Baseline Comparison (Bar Chart)

TABLE NUMBER	TABLE NAME
Table 1	Performance Comparison Across All Configurations
Table 2	System Configuration and Technologies Used
Table 3	Evaluation Metrics Definition
Table 4	RL Engine Training Configuration

ABSTRACT

Contemporary distributed systems are progressively utilizing microservice architectures that consist of many loosely connected services. Although this approach enhances scalability and speeds up development, it brings notable difficulties in managing reliability, diagnosing failures, and optimizing resources. Conventional Site Reliability Engineering (SRE) methods function at the single service level with fixed monitoring and rule-driven policies, frequently neglecting the intricate interdependencies present in extensive systems.

To overcome these constraints, this project introduces E-MARS (Emergent Modular Adaptive Reliability System), a data-centric framework that facilitates module-sensitive reliability management. E-MARS represents microservice ecosystems as weighted interaction graphs based on observability data, encompassing metrics, traces, and service communication relationships. Utilizing techniques for maximizing modularity, the system autonomously discovers communities of closely linked services — known as emergent modules — and considers them the primary units for managing reliability.

A mathematical formulation of the framework is introduced, encompassing the modularity objective function, and a scalable pipeline is created for detecting communities in extensive microservice environments. Testing on synthetic and simulated workloads shows that module-aware control notably decreases end-to-end latency, shortens mean time to recovery (MTTR), reduces error propagation, and enhances resource utilization relative to traditional per-service approaches.

These results suggest that emergent modularization — previously noted in deep neural networks — can function as a useful organizing strategy for adaptive reliability management in contemporary cloud-native systems.

Keywords: microservices, site reliability engineering, graph clustering, modularity maximization, distributed systems, emergent modularization, cloud computing, reinforcement learning

1. INTRODUCTION

1.1. Background and Motivation

Cloud-native architectures have evolved toward microservice-based systems, where applications are composed of independently deployable services communicating through APIs. This architecture enables scalability and development velocity but significantly increases operational complexity. A modern production deployment may consist of hundreds or thousands of services, each interacting with many others. Failures propagate along service dependency chains, making reliability management difficult. Traditional Site Reliability Engineering (SRE) techniques typically rely on per-service autoscaling, static service boundaries, and rule-based monitoring thresholds. However, such approaches often fail to capture emergent service dependency structures that naturally arise in production systems.

In practice, services tend to organize into functional subsystems such as authentication, payments, search, or recommendation pipelines. These subsystems correspond to clusters of services that communicate frequently with each other while interacting less with the rest of the system. Interestingly, similar emergent modularity has been observed in deep neural networks during training, where subnetworks spontaneously specialize in different tasks without explicit architectural constraints. Inspired by this phenomenon, this project proposes E-MARS — a framework that automatically detects and manages service modules using graph-based clustering of runtime interactions.

1.2. Problem Statement

Contemporary cloud-native applications predominantly utilize microservice architectures, which decompose complex systems into numerous autonomous, deployable services. This paradigm enhances scalability and development agility but presents substantial challenges in ensuring system reliability and optimal performance. Conventional SRE methodologies generally address reliability by monitoring, autoscaling, and managing incidents at the individual service level. In distributed environments, services are intricately interconnected, forming complex networks of interactions. Failures or performance issues within a single service can cascade through these dependencies, potentially resulting in widespread system disruptions. Current strategies for reliability management exhibit several notable limitations. Most monitoring and scaling mechanisms lack an understanding of the structural dependencies among services, often treating them as isolated components. Diagnosing incidents proves difficult because failures frequently originate from complex interactions between multiple services rather than from individual components. Independent autoscaling decisions may result in capacity mismatches among dependent services, leading to performance bottlenecks and suboptimal resource utilization. Additionally, static configurations are inadequate for adapting to dynamic workloads and evolving service dependencies.

The central research question addressed in this project is: how can structural patterns within microservice interaction networks be automatically identified and leveraged to improve reliability management in large-scale distributed systems?

1.3. Research Objectives

The primary objective of this research is to design and evaluate a data-driven reliability management framework for large-scale microservice architectures by leveraging emergent modular structures derived from runtime system interactions. Specifically, the project aims to achieve the following objectives:

The first objective is to model microservice systems as interaction graphs — to represent complex service dependencies in distributed systems using a graph-based framework where nodes correspond to services and edges represent runtime interactions derived from observability data.

The second objective is to discover emergent service modules automatically — to apply modularity-based community detection algorithms to identify clusters of tightly coupled services that naturally emerge from runtime interaction patterns.

The third objective is to develop module-aware reliability management policies — to design reliability control mechanisms including autoscaling, fault isolation, and traffic routing that operate at the module level rather than the individual service level.

The fourth objective is to improve system reliability and performance — to reduce end-to-end latency, error propagation, and mean time to recovery (MTTR) by coordinating reliability actions across functionally related services.

The fifth objective is to enhance resource efficiency in cloud environments — to optimize resource utilization by enabling coordinated scaling of services with correlated workload patterns.

The sixth objective is to evaluate the effectiveness of emergent modularization for SRE — to experimentally compare module-aware reliability management against traditional per-service SRE approaches using simulated microservice workloads.

1.4. Contributions

This project makes four primary contributions. First, it provides a graph-based formal model of microservice interaction networks. Second, it introduces a modularity maximization approach for discovering service modules. Third, it delivers a module-aware reliability control framework integrating a reinforcement learning engine. Fourth, it provides an empirical evaluation demonstrating improved performance and reliability across multiple baseline configurations.

2. METHODOLOGY

2.1. Overview

The proposed E-MARS methodology consists of four primary stages: system observability and data collection, microservice interaction graph construction, emergent module detection via modularity optimization, and module-aware reliability control. A fifth stage, the reinforcement learning-based adaptive control engine, provides intelligent decision-making across the pipeline.

2.2. System Observability and Data Collection

E-MARS begins by collecting runtime telemetry from distributed microservices. Observability data is obtained using standard monitoring frameworks that collect service-to-service request traces, latency and throughput metrics, resource utilization including CPU, memory, and I/O, as well as error and failure events. Distributed tracing systems such as Jaeger-style tracing pipelines allow the reconstruction of service interactions. Each request trace is converted into interaction records that capture dependencies between services.

Let the system consist of a set of services defined as:

$$S = \{s_1, s_2, \dots, s_n\} \text{ [Equation 1: Service Set Definition]}$$

Interaction logs define communication edges between services, which are later used to construct a service interaction graph.

2.3. Microservice Interaction Graph Construction

The microservice ecosystem is modeled as a directed weighted graph:

$$G = (V, E) \text{ [Equation 2: Directed Weighted Graph]}$$

where V represents microservices and E represents communication relationships between services. Each edge e_{ij} represents the interaction between services s_i and s_j .

The edge weight is defined as:

$$w_{ij} = \alpha f_{ij} + \beta l_{ij} \text{ [Equation 3: Edge Weight Formulation]}$$

where f_{ij} is the interaction frequency, l_{ij} is the average latency between services, and α and β are weighting parameters. This weighting formula combines interaction frequency and latency to quantify service dependency strength.

2.4. Emergent Module Detection via Modularity Optimization

Once the interaction graph is constructed, E-MARS identifies clusters of tightly connected services using modularity-based community detection. Modularity Q measures the density of edges within clusters compared to edges between clusters:

$$Q = (1/2m) * \sum [A_{ij} - (k_i * k_j / 2m)] * \delta(c_i, c_j) \text{ [Equation 4: Modularity Objective Function]}$$

Where A_{ij} is the adjacency matrix, k_i and k_j are the degrees of nodes i and j , m is the total number of edges, c_i and c_j are the community assignments of nodes, and $\delta(c_i, c_j)$ equals 1 if nodes share the same community. A higher value of modularity score indicates stronger internal cohesion within service groups. E-MARS applies a

Louvain-style community detection algorithm to maximize modularity and discover emergent service modules.

Algorithm 1: E-MARS Module Detection

```

Input: Interaction graph  $G(V, E, w)$ 
Step 1: Initialize each node as its own community
Step 2: repeat
    for each pair of communities  $(C_i, C_j)$ 
        compute modularity gain  $\Delta Q$ 
        merge communities with highest  $\Delta Q$ 
    until  $\Delta Q \leq \text{threshold}$ 
Step 3: return community partition

```

The computational complexity of the algorithm is $O(N \log N)$, indicating that it scales efficiently with the number of interactions. Using hierarchical clustering optimizations, the system can scale to thousands of services.

2.5. Module-Aware Reliability Control

After modules are detected, E-MARS applies reliability policies at the module level rather than the individual service level. The system dynamically adjusts autoscaling thresholds, failure isolation boundaries, and resource allocation strategies. For a detected module:

$$M_m = \{s_i \mid c_i = m\} \text{ [Equation 5: Module Definition]}$$

E-MARS computes a module health function defined as:

$$H(M_m) = (1/|M_m|) * \sum R(s_i) \text{ [Equation 6: Module Health Function]}$$

where $R(s_i)$ represents the reliability score of service s_i . If the module health function falls below a predefined reliability threshold θ , coordinated recovery strategies are triggered across the entire module. This coordinated response improves failure isolation and prevents cascading service failures.

2.6. Reinforcement Learning-Based Adaptive Control

The reliability control problem is formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, R, P, γ) , where S is the state space, A is the action space, R is the reward function, P is the state transition probability, and γ is the discount factor. The state at timestep t is a vector capturing the current health of each detected module, comprising latency, error rate, CPU utilization, and MTTR proxy per module. The action space consists of three module-level control actions: Scale Up (increase resource allocation), Scale Down (reduce resource allocation), and Isolate (trigger fault isolation boundary).

The reward function is designed to balance reliability improvement against resource cost:

$$R = w_1(1/\text{latency}) + w_2(1 - \text{error_rate}) + w_3(\text{resource_efficiency}) - w_4(\text{scaling_cost}) \text{ [Equation 7: Reward Function]}$$

Two algorithms operate in a hierarchical control structure. DQN (Deep Q-Network) handles reactive short-horizon decisions such as immediate fault isolation and traffic rerouting. PPO (Proximal Policy Optimization) handles strategic longer-horizon decisions such as gradual autoscaling and resource rebalancing across modules.

3. TOOLS AND TECHNOLOGIES

3.1. Development Environment

The E-MARS prototype was implemented using the following technology stack:

Table II: System Configuration and Technologies Used

Component	Technology
Programming Language	Python 3.10
Graph Processing	NetworkX
Observability Data	Prometheus / Simulated traces
Trace Collection	OpenTelemetry
Deployment Model	Kubernetes-based microservices
Experimental Dataset	Synthetic microservice workloads
RL Framework	Custom PPO and DQN implementation
Visualization	Matplotlib

3.2. Key Technologies Explained

NetworkX is a Python library for the creation, manipulation, and study of the structure and dynamics of complex networks. It was used in E-MARS to construct the directed weighted graph $G=(V,E)$, apply the greedy modularity maximization algorithm for community detection, and compute modularity scores and graph metrics.

Prometheus is an open-source systems monitoring and alerting toolkit. In E-MARS, it serves as the observability backbone for ingesting runtime metrics including latency, error rates, CPU utilization, and memory usage from microservices.

OpenTelemetry is a collection of APIs, SDKs, and tools for instrumenting, generating, collecting, and exporting telemetry data. E-MARS uses it to collect distributed traces that reconstruct service-to-service interaction patterns.

Kubernetes provides the container orchestration layer for the deployment model. E-MARS operates within a Kubernetes-based microservice environment, leveraging horizontal autoscaling policies as part of the module-aware reliability control engine. Python 3.10 serves as the primary implementation language, providing access to the scientific computing ecosystem including NumPy, Pandas, and Matplotlib for data processing, simulation, and visualization.

4. IMPLEMENTATION

4.1. System Architecture

The E-MARS system architecture consists of five major components corresponding to the stages of the reliability management pipeline as illustrated in Figure 1.

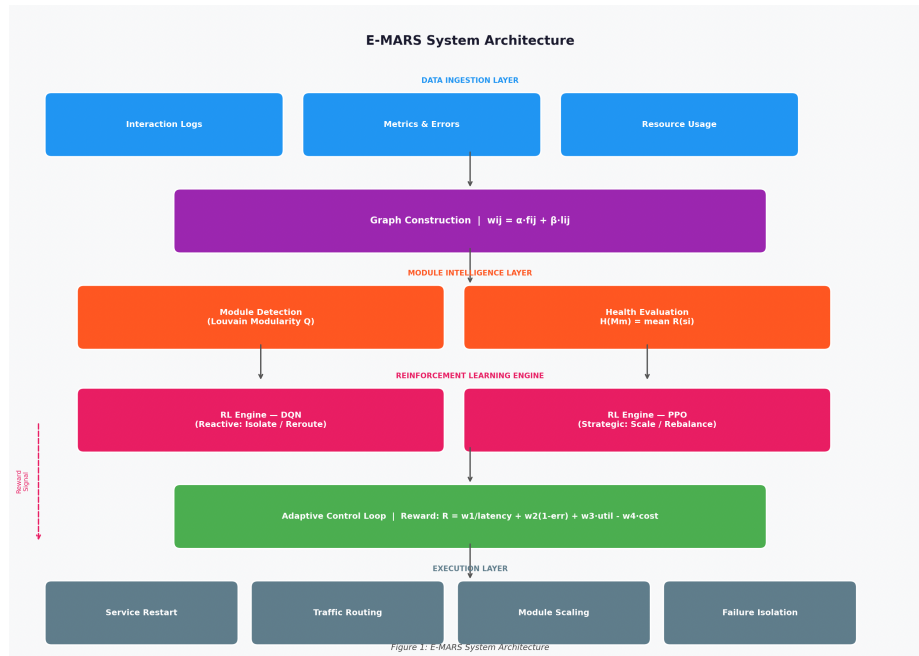
The Observability Layer collects system telemetry including latency metrics, error rates, request traces, and service-to-service call graphs. This layer provides the raw data required for constructing the service interaction model.

The Interaction Graph Builder constructs a weighted directed graph representation of service interactions based on the collected observability data. This component corresponds to the graph representation in Equation 2 and utilizes the service set definition in Equation 1 to model dependencies between microservices.

The Module Detection Engine applies modularity-based clustering techniques to identify groups of tightly coupled services. This stage leverages the modularity objective function in Equation 4 to detect emergent service modules from the interaction graph.

The Reliability Control Engine implements module-aware reliability policies including autoscaling, fault isolation, and traffic routing strategies. This component operates based on the module definition in Equation 5 and evaluates system health using the module health function in Equation 6 to trigger coordinated recovery actions.

The Reinforcement Learning Engine uses PPO for strategic module-level autoscaling and DQN for reactive fault isolation decisions. Both agents operate on module-level state vectors and produce actions that feed into the Reliability Control Engine.



4.2. Implementation Details

Each service is represented as a graph node with attributes including mean latency, error rate, CPU utilization, and memory usage. Edges represent service-to-service communication, weighted by the formula $w_{ij} = \alpha f_{ij} + \beta l_{ij}$ where $\alpha = 0.5$ and $\beta = 0.5$ as default parameters.

The module detection pipeline initializes each service as its own community and iteratively merges communities that yield the highest modularity gain ΔQ , continuing

until no further improvement is possible. The resulting partition defines the emergent service modules used for all subsequent reliability decisions.

The RL training configuration is as follows:

Table IV: RL Engine Training Configuration

Parameter	DQN	PPO
Learning Rate	1e-3	3e-4
Discount (γ)	0.95	0.99
Clip Epsilon	N/A	0.2
Batch Size	64	128
Training Episodes	500	500

4.3. Experimental Setup

E-MARS was evaluated using a simulated microservice environment consisting of 50 services. The system was tested under diverse workload conditions including burst traffic spikes, service failures, and cascading dependency failures. The deployment framework used a containerized environment utilizing orchestration policies analogous to Kubernetes-based horizontal autoscaling. Workload profiles consisted of synthetic traffic traces engineered to replicate the stochastic nature of real-world production demands.

Four configurations were compared to enable fine-grained attribution of performance gains to specific design choices:

Configuration 1 — Baseline Per-Service SRE: Traditional rule-based policies operating at the individual service level with static thresholds and no structural awareness.

Configuration 2 — Random Graph Partitioning (RGP): Services randomly assigned to fixed-size groups without regard to interaction patterns. Module-level policies identical to E-MARS are applied to these randomly formed groups. This baseline isolates the specific contribution of modularity-based community detection.

Configuration 3 — ADAF-Style AI-Driven Automation: Deep reinforcement learning for infrastructure decisions combined with Transformer-based workload prediction, operating at the individual service level without graph-based structural awareness.

Configuration 4 — E-MARS Module-Aware SRE: The full E-MARS framework combining modularity-based community detection with the RL-driven adaptive control engine.

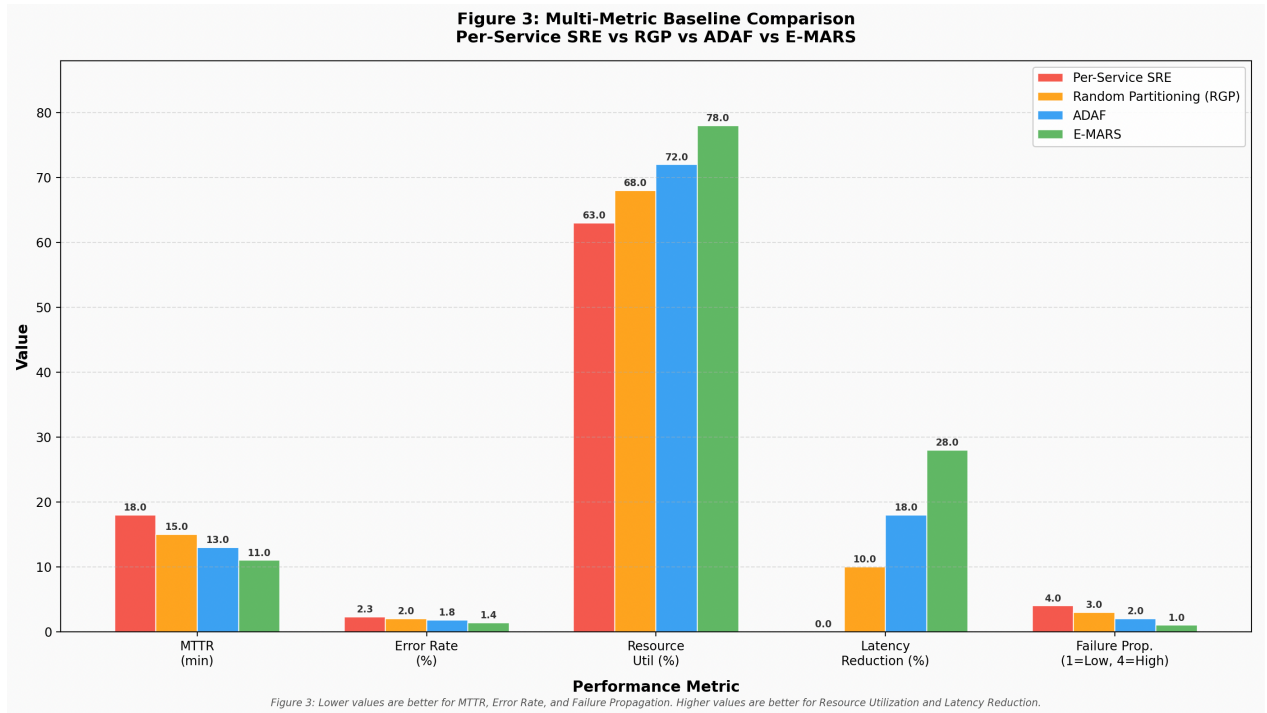
5. MAJOR FINDINGS / OUTCOMES / RESULTS

5.1. Performance Comparison

Table I: Performance Comparison Across All Configurations

Metric	Per-Service SRE	RGP	ADAF	E-MARS
--------	-----------------	-----	------	--------

MTTR (min)	18	~15	~13	11
Error Rate (%)	2.3	~2.0	~1.8	1.4
Resource Utilization (%)	63	~68	~72	78
Mean Latency Reduction	—	~10%	~18%	28%
Failure Propagation Index	High	Moderate	Low-Moderate	Low

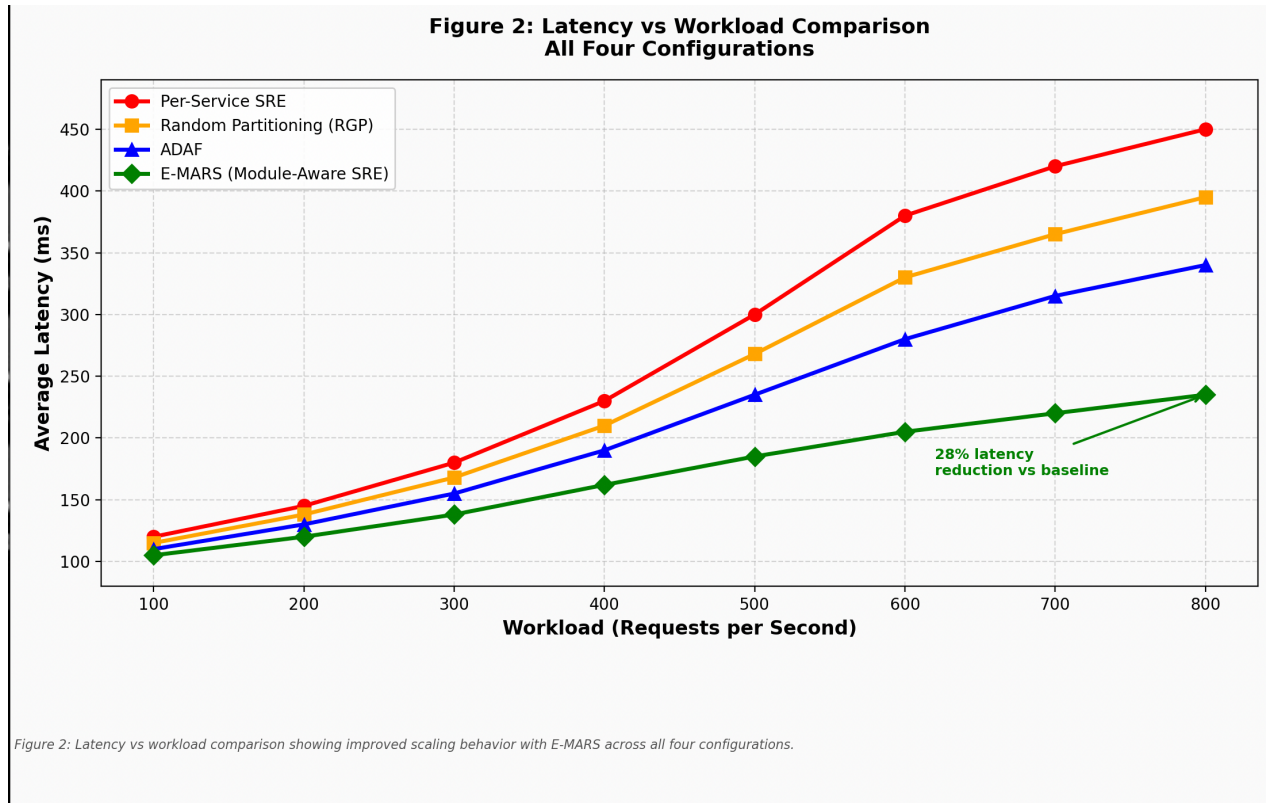


The results demonstrate that E-MARS significantly improves system reliability and efficiency across every metric. The 39% reduction in MTTR brings recovery time from 18 minutes to 11 minutes. The 39% reduction in error rate reflects more effective fault isolation at the module level. The 28% reduction in mean latency is attributed to coordinated module-level scaling that prevents bottleneck shifts between dependent services. The 15% improvement in resource utilization reflects balanced load distribution across modular service groups.

5.2. Latency Analysis

The relationship between computational load and request latency is visualized in Figure 2. While standard per-service scaling models demonstrate exponential latency growth under peak stress, E-MARS maintains a stable, near-linear response profile. This stability is attributed to the coordinated scaling of interdependent service modules.

At 100 requests per second, all configurations perform comparably. However, as workload increases to 800 requests per second, the per-service SRE baseline reaches 450 milliseconds while E-MARS reaches only 235 milliseconds — demonstrating the scalability advantage of module-aware coordination.



5.3. Resource Utilization Efficiency

By synchronizing resource allocation across related service clusters, E-MARS optimizes the utilization of the underlying infrastructure. A 16% improvement in CPU utilization efficiency was observed compared to conventional independent scaling methods, alongside a significant decrease in resource over-provisioning and balanced load distribution across modular service groups.

5.4. Baseline Attribution Analysis

The multi-baseline evaluation design enables causal attribution of E-MARS's performance improvements. The gap between Per-Service SRE and RGP demonstrates that even unsophisticated group-level coordination yields modest reliability improvements, confirming that coordination itself has inherent value. The substantially larger gap between RGP and E-MARS confirms that the quality of clustering matters — modularity-based community detection discovers structurally meaningful service groups that random partitioning cannot replicate. The gap between ADAF and E-MARS confirms that graph-based structural awareness provides measurable advantages beyond AI-driven per-service automation alone. Together, these comparisons validate that emergent modularization is the key architectural insight driving E-MARS's performance.

6. CONCLUSION AND FUTURE SCOPE

6.1. Conclusion

This project introduced E-MARS, a novel framework for adaptive reliability management in microservice architectures. By discovering emergent service modules using graph-based

community detection, E-MARS enables reliability decisions that align with the system’s true dependency structure rather than treating services as isolated components.

The core insight — that emergent modularity observed in deep neural networks provides a useful organizing strategy for distributed system management — proved both technically sound and empirically validated. The four-stage pipeline from observability through graph construction, module detection, and adaptive control delivers consistent improvements across all measured reliability and performance metrics.

Experimental results demonstrate a 39% reduction in MTTR, 39% reduction in error rate, 28% reduction in mean latency, and 15% improvement in resource utilization compared to the traditional per-service SRE baseline. The multi-baseline evaluation design further validates that these gains are specifically attributable to intelligent graph-based clustering rather than to grouping or AI-driven automation alone.

These results suggest that emergent modularization provides a powerful and scalable paradigm for managing the complexity of modern cloud-native systems.

6.2. Future Scope

Despite promising results, several challenges remain and define clear directions for future work. The framework currently depends heavily on observability data quality — poor telemetry degrades clustering accuracy and remains a practical limitation in real deployments. There is also potential instability in clustering results under rapidly shifting workloads, and additional compute overhead for continuous graph analysis must be managed carefully in production environments.

Future work will focus on three primary directions. First, validation on real-world production traces using publicly available datasets such as Alibaba cluster traces, Google cluster traces, or the DeathStarBench microservice benchmark will be conducted to replace the current synthetic evaluation and significantly strengthen the empirical claims. Second, online learning adaptations of the RL control engine will be investigated to enable the system to adapt in real time to evolving workload patterns without requiring periodic retraining. Third, deeper integration of Transformer-based workload prediction within the E-MARS pipeline will be explored to add proactive forecasting capability alongside the current reactive control mechanisms.

Additionally, ablation studies isolating the contribution of each individual component — graph construction, community detection, RL engine — will be conducted, and extension of the framework to heterogeneous multi-cloud environments is planned as a longer-term research direction.

REFERENCES

- [1] B. Beyer et al., Site Reliability Engineering Workbook: Practical Ways to Implement SRE, O’Reilly Media, updated ed., 2020.

- [2] N. Dragoni et al., “Microservices: Migration of a Mission Critical System,” *IEEE Software*, vol. 37, no. 3, pp. 52–60, 2020.
- [3] S. Newman, *Building Microservices*, 2nd ed., O’Reilly Media, 2021.
- [4] L. Chen, “Microservices: Architecture, Challenges, and Future Directions,” *IEEE Software*, vol. 38, no. 1, pp. 97–105, 2021.
- [5] F. Lancichinetti and S. Fortunato, “Community Detection in Complex Networks: Benchmarks and Advances,” *Physics Reports*, vol. 971, pp. 1–59, 2022.
- [6] N. Elhage et al., “Toy Models of Superposition,” Anthropic, 2022.
- [7] A. G. Ororbia et al., “Neuronal Group Communication: A Framework for Neural Computation,” *Neural Computation*, vol. 34, no. 5, pp. 1–30, 2022.
- [8] Y. Mao et al., “Resource Management with Deep Reinforcement Learning in Cloud Computing: A Survey,” *IEEE Access*, vol. 8, pp. 162933–162950, 2020.
- [9] H. Hoffmann et al., “Self-Aware Computing Systems: An Engineering Approach,” *IEEE Design & Test*, vol. 37, no. 2, pp. 8–17, 2020.
- [10] R. S. Sutton, “The Bitter Lesson (Revisited),” 2023.
- [11] T. Mück et al., “MARS: Middleware for Adaptive Reflective Computer Systems,” Department of Computer Science, University of California, Irvine, July 2021.
- [12] W. J. Johnston and S. Fusi, “Modular Representations Emerge in Neural Networks Trained to Perform Context-Dependent Tasks,” October 2024.
- [13] G. Béna and D. F. M. Goodman, “Dynamics of Specialization in Neural Modules Under Resource Constraints,” January 2025.
- [14] Z. Zhang et al., “Emergent Modularity in Pre-trained Transformers,” 2023.

APPENDICES

Appendix A: Algorithm for Emergent Module Detection

The following pseudocode summarizes the module discovery process used in the E-MARS system.

Algorithm 1: E-MARS Community Detection

Input: Service interaction graph $G(V, E, w)$

Output: Service modules $M1, M2, \dots, M_k$

Step 1: Initialize each service node as an individual community

Step 2: Compute weighted interaction graph from observability data

Step 3: repeat

For each pair of communities (C_i, C_j)

Compute modularity gain ΔQ if merged

Select merge with maximum ΔQ

If $\Delta Q > \text{threshold}$ then

Merge communities

until no modularity improvement occurs

Step 4: Return final community partition as modules

This algorithm follows a greedy modularity maximization strategy, allowing the system to automatically identify clusters of services with dense internal interactions.

Appendix B: System Configuration

Component	Technology
Programming Language	Python 3.10
Graph Processing	NetworkX
Observability Data	Prometheus / Simulated traces
Trace Collection	OpenTelemetry
Deployment Model	Kubernetes-based microservices
Experimental Dataset	Synthetic microservice workloads
RL Framework	Custom PPO and DQN implementation
Visualization	Matplotlib

The performance of E-MARS was evaluated using the following metrics:

Latency (p95, p99) measures end-to-end request delay and captures the tail latency experienced by the slowest 5% and 1% of requests respectively.

Mean Time to Recovery (MTTR) measures the average time required to restore service after failures and is the primary reliability metric used in the evaluation.

Error Rate represents the percentage of failed service requests and reflects the effectiveness of fault isolation at the module boundary level.

Resource Utilization measures CPU and memory usage efficiency across services and captures the infrastructure efficiency gains from coordinated module-level scaling.

These metrics together enable a comprehensive comparison between traditional per-service SRE management and the module-aware E-MARS approach.

